



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Shubham Kumar
May 13, 2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
 - Data Collection
 - Data wrangling
 - Exploratory Data Analysis (EDA) with matplotlib/seaborn and SQL
 - Interactive map built with Folium
 - Dashboard built with Plotly Dash
 - Predictive analysis with Classification models
- Summary of all results
 - EDA results
 - Interactive analytics
 - Predictive analysis

Introduction

- Project background and context
 - SpaceX aims to reduce the cost of space travel through reusable launch vehicles. Understanding the factors influencing successful launches helps improve planning and efficiency.
- Problems you want to find answers
 - Which launch sites have the highest success rates?
 - What payload masses tend to be successful?
 - Can we predict the outcome of a future launch based on features like orbit, mass, and site?

Section 1

Methodology



Methodology

Executive Summary

- Data collection methodology:
 - SpaceX REST API with *requests*.
 - Web Scraping from Wikipedia using *BeautifulSoup* and *pandas*.
- Perform data wrangling
 - Merged API and scraped data; One Hot Encoding data fields for Machine Learning
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection

Data sets are collected using two methods:

1. Using the SpaceX API

- Launch data was collected from the official SpaceX Launches API.

2.Web Scraping

- Launch site geographical data (latitude and longitude) was scraped from Wikipedia's SpaceX Launch Sites.

Data Collection – SpaceX API

1. Get Response from SpaceX API



2. Parse JSON



3. Perform pre-processing by updating columns and rows with functions



4. Assign list to Dictionary and Create pandas DataFrame



5. Filter to keep Falcon 9 launches and Export as .csv

1.

```
In [7]: spacex_url="https://api.spacexdata.com/v4/launches/past"

In [8]: response = requests.get(spacex_url)
```

2.

```
In [12]: # Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

3.

```
# Call getBoosterVersion
getBoosterVersion(data)
```

```
# Call getLaunchSite
getLaunchSite(data)
```

```
# Call getPayloadData
getPayloadData(data)
```

```
# Call getCoreData
getCoreData(data)
```

4.

```
In [22]: launch_dict = {'FlightNumber': list(data['flight_number']),
                        'Date': list(data['date']),
                        'BoosterVersion': BoosterVersion,
                        'PayloadMass': PayloadMass,
                        'Orbit': Orbit,
                        'LaunchSite': LaunchSite,
                        'Outcome': Outcome,
                        'Flights': Flights,
                        'GridFins': GridFins,
                        'Reused': Reused,
                        'Legs': Legs,
                        'LandingPad': LandingPad,
                        'Block': Block,
                        'ReusedCount': ReusedCount,
                        'Serial': Serial,
                        'Longitude': Longitude,
                        'Latitude': Latitude}
```

```
In [23]: # Create a data from launch_dict
dataframe = pd.DataFrame.from_dict(launch_dict)
```

5.

```
In [26]: # Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = dataframe[dataframe['BoosterVersion']!='Falcon 1']

In [35]: data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

Data Collection - Scraping

1. Get Response from HTML and Create BeautifulSoup Object



2. Find tables and get column names



3. Create Dictionary



4. Append data to keys



5. Convert dictionary to dataframe and export to .csv

1.

```
In [8]: # use requests.get() method with the provided static_url
response = requests.get(static_url)
```

```
In [11]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.text, 'html.parser')
```

2.

```
In [13]: # Use the find_all function in the BeautifulSoup object, with element type 'table'
html_tables = soup.find_all('table')
# Assign the result to a list called 'html_tables'
```

```
In [16]: column_names = []

# Apply find_all() function with 'th' element on first_launch_table
th_elements = first_launch_table.find_all('th')
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
for th_element in th_elements:
    name = extract_column_from_header(th_element)

# Append the Non-empty column name ('if name is not None and len(name) > 0') into a list called column_names
if name is not None and len(name) > 0:
    column_names.append(name)
```

3.

```
In [18]: launch_dict= dict.fromkeys(column_names)

# Remove an irrelevant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty List
launch_dict['Flight No.']= []
launch_dict['Launch site']= []
launch_dict['Payload']= []
launch_dict['Payload mass']= []
launch_dict['Orbit']= []
launch_dict['Customer']= []
launch_dict['Launch outcome']= []

# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]
```

4.

```
In [25]: extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table','wikitable plainrowheaders collapsible')):
    # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
            #get table element
            row=rows.find_all("td")
            #if it is number save cells in a dictionary
            if flag:
                extracted_row += 1
                # Flight Number value
                launch_dict['Flight No.'].append(flight_number)
                print(flight_number)
                datetimelist=date_time(row[0])

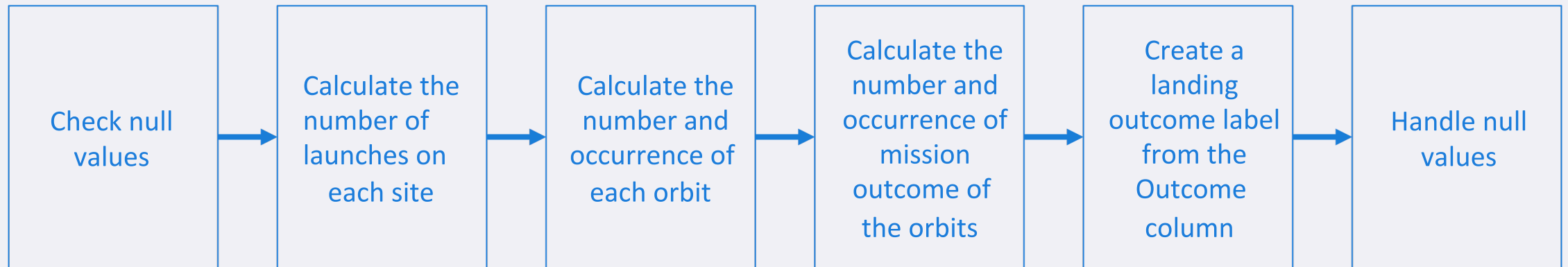
            # Date value
            launch_dict['Date'].append(date)
```

5.

```
In [26]: df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

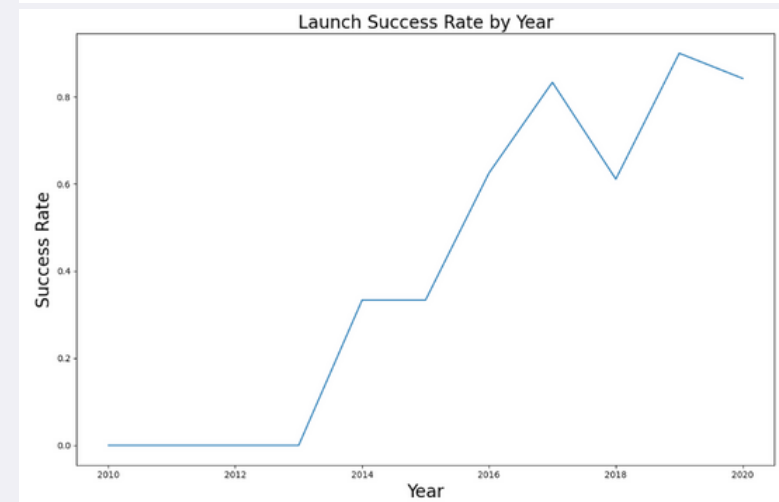
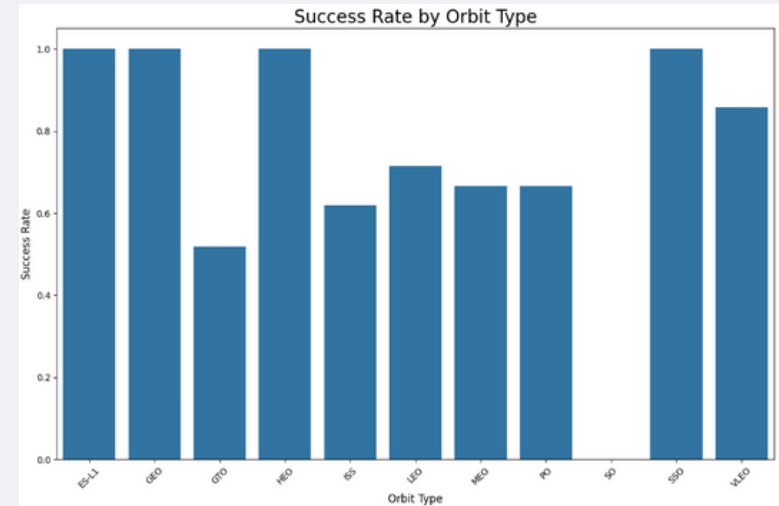
```
In [27]: df.to_csv('spacex_web_scraped.csv', index=False)
```

Data Wrangling



EDA with Data Visualization

- Plotted Flight Number vs Launch Site and Payload Mass vs Launch Site to explore success trends.
- Visualized Orbit Type success rates using bar charts.
- Used seaborn/matplotlib to identify potential correlations between orbit, payload mass, booster version, and success.
- Extracted launch years to visualize yearly trends.



EDA withSQL

- Used SQLite to query structured launch dataset.
- Sample queries included:
 - Unique launch site names
 - Success counts grouped by orbit type
 - Booster versions with highest payload mass
 - First successful landings

```
In [30]: %sql SELECT Landing_Outcome, COUNT(Landing_Outcome) as Count FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-01'
```

* sqlite:///my_data1.db
Done.

Out[30]:

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Build an Interactive Map with Folium

- Used Folium to visualize SpaceX launch sites on an interactive map.
- Added Marker objects with tooltips showing launch pad names and results.
- Calculated marker colors based on launch outcome: green (success), red (failure).
- Added Circle markers to highlight regions of repeated launches.

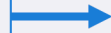
Build a Dashboard with Plotly Dash

- Built an interactive dashboard using Plotly Dash with:
 - A site filter dropdown for launch sites
 - A pie chart showing launch success counts
 - A payload mass range slider that filters scatter plots
 - A scatter plot showing relationship between payload and success by booster version

Predictive Analysis (Classification)

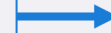
1. Building the model

- Create column for the class
- Standardize the data
- Split the data into train and test sets
- Build GridSearchCV model and fit the data



2. Evaluating the model

- Calculating the accuracies
- Calculating the confusion matrix
- Plot the results



3. Finding the optimal model

- Find the best hyperparameters for the model
- Find the best model with the highest accuracy
- Confirm the optimal model

Results

- Exploratory Data Analysis

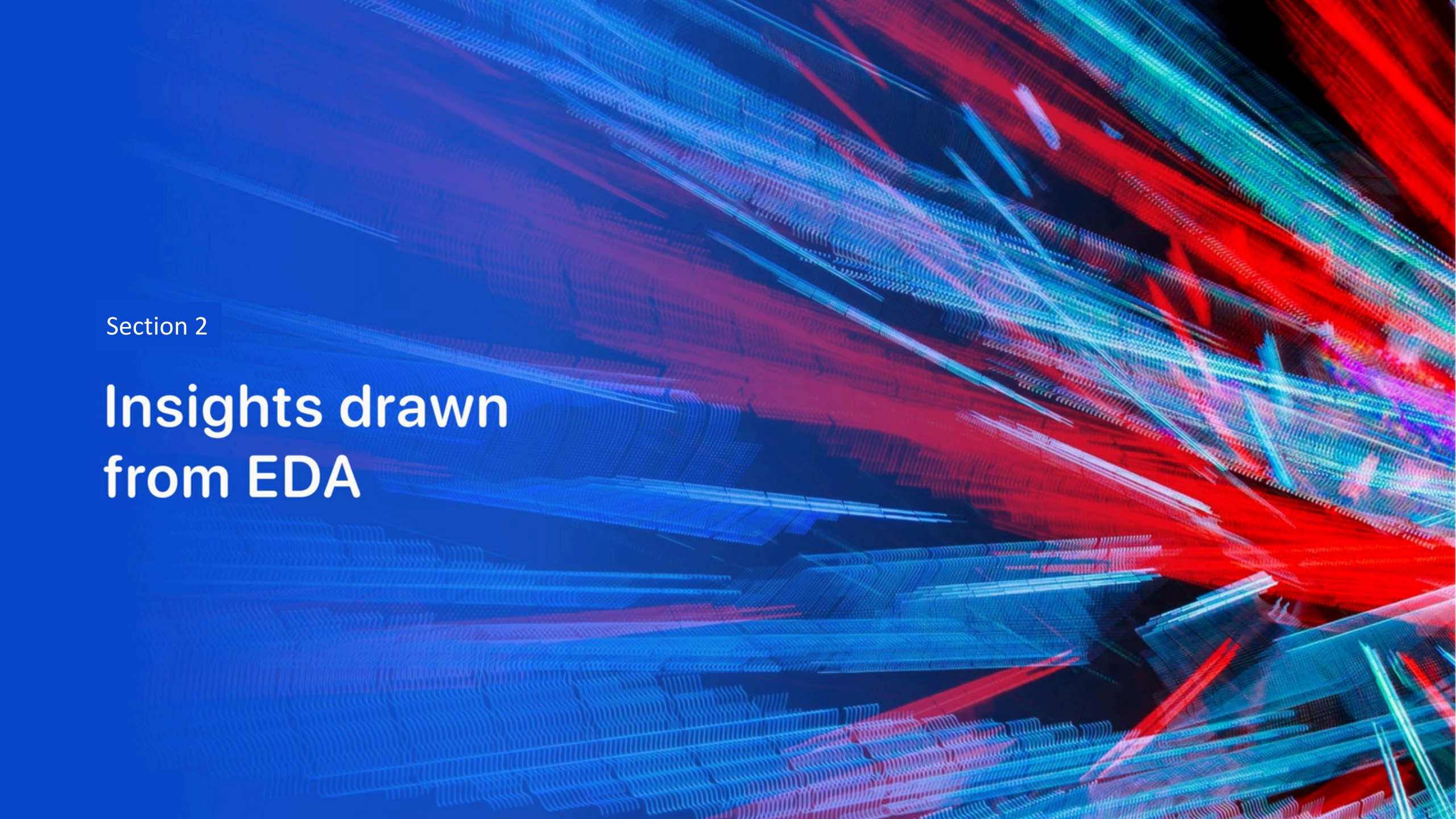
- LEO missions have the highest success rates
- Mid-weight payloads (~4 000–6 000 kg) tend to succeed more often
- FT boosters outperform other versions in raw success count

- Interactive Analytics

- Folium map reveals geographic clustering and infrastructure proximity
- Dash dashboard lets users filter by site & payload to spot patterns in real-time

- Predictive Modeling

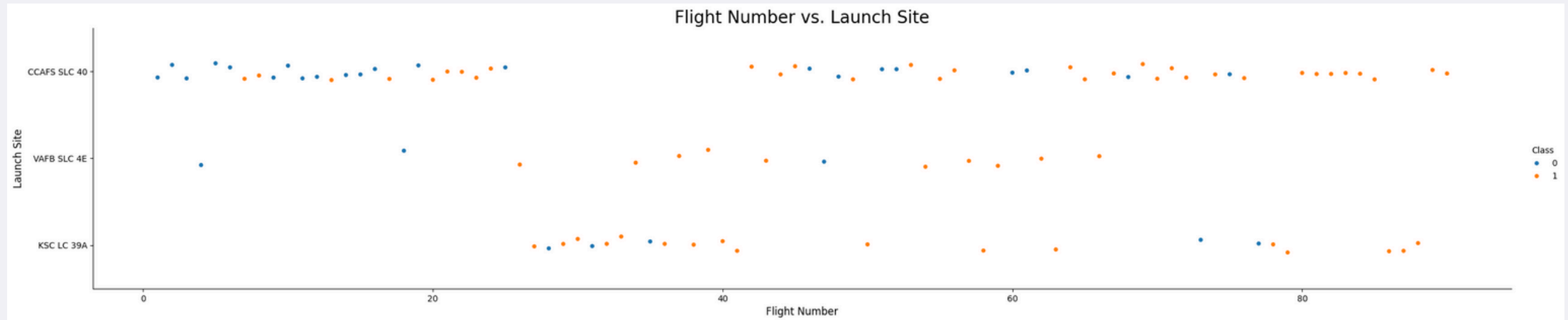
- Logistic Regression reached 83.3% accuracy on the test set
- Confusion matrix: 10/12 correct, most errors on border-case payloads

The background of the slide is an abstract composition. It features a solid blue area on the left side, which transitions into a complex pattern of diagonal streaks and lines in shades of blue, red, and cyan on the right. These streaks have a textured, almost woven appearance, suggesting a digital or data-driven theme. The overall effect is dynamic and modern.

Section 2

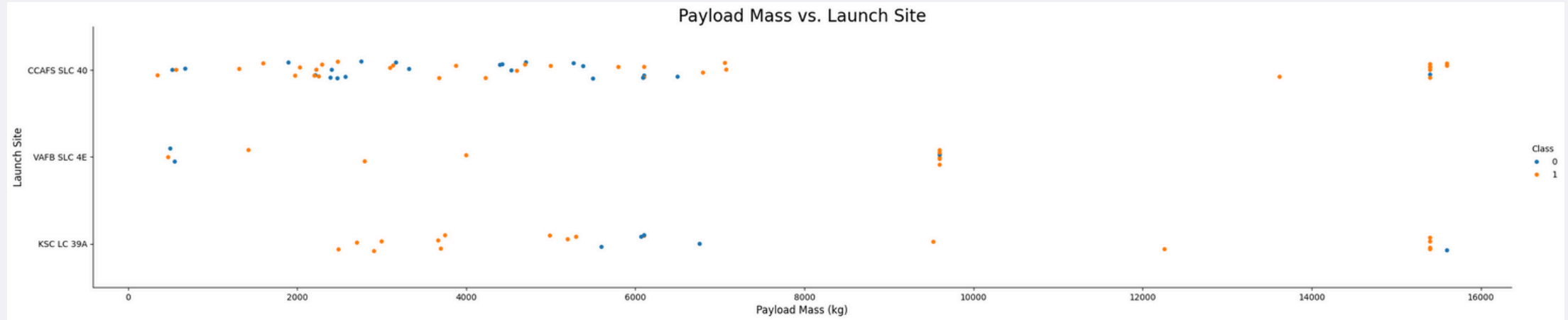
Insights drawn from EDA

Flight Number vs. LaunchSite



The success rate of launches increases as the number of Flights increase at each Launch Site

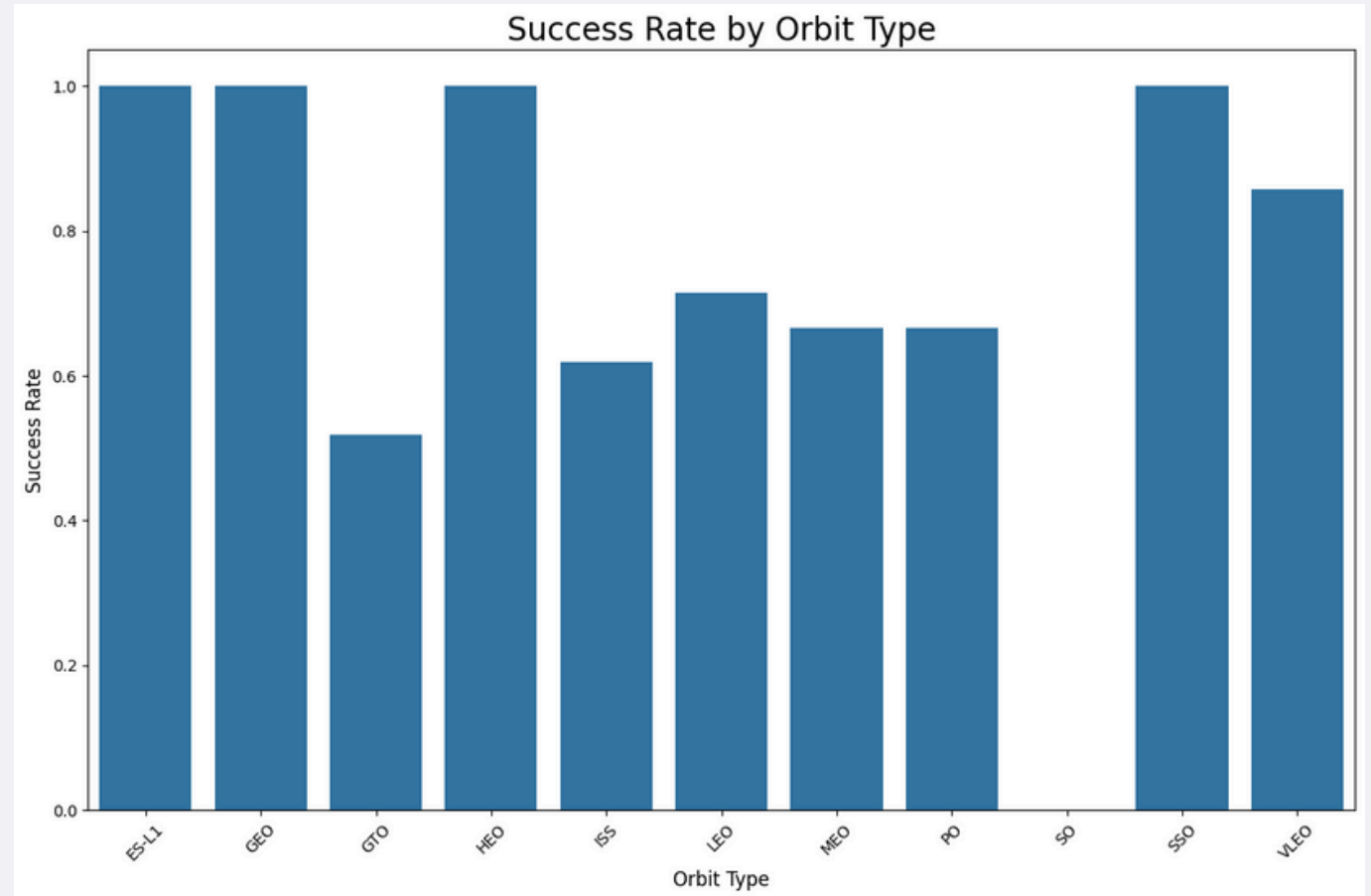
Payload vs. Launch Site



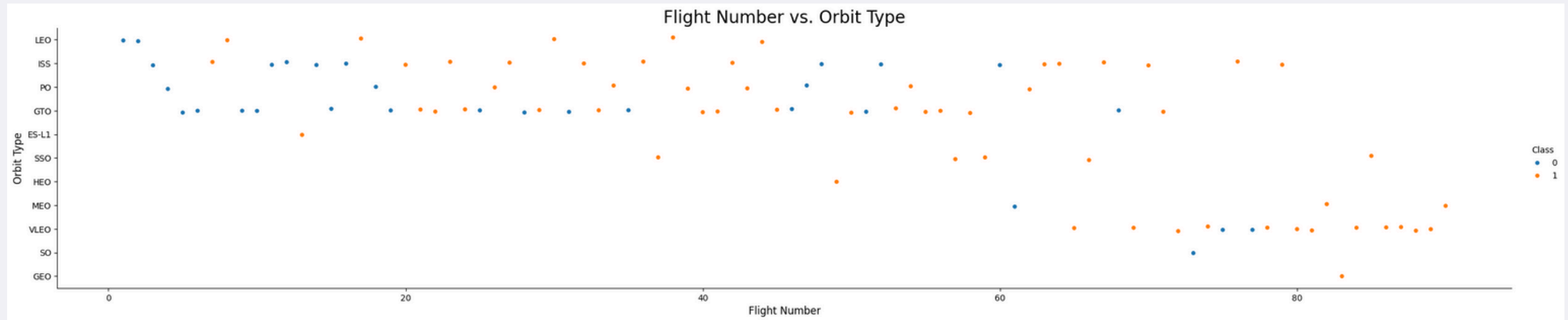
As Payload Mass increases above 8000 kg so does the success rate of launches at all three launch sites. Although for the VAFB-SLC launch site there are no rockets launched for heavy payload mass (greater than 10000)

Success Rate vs. Orbit Type

- ES-L1, GEO, HEO, and SSO have a success rate of 100%
- SO has a success rate of 0%

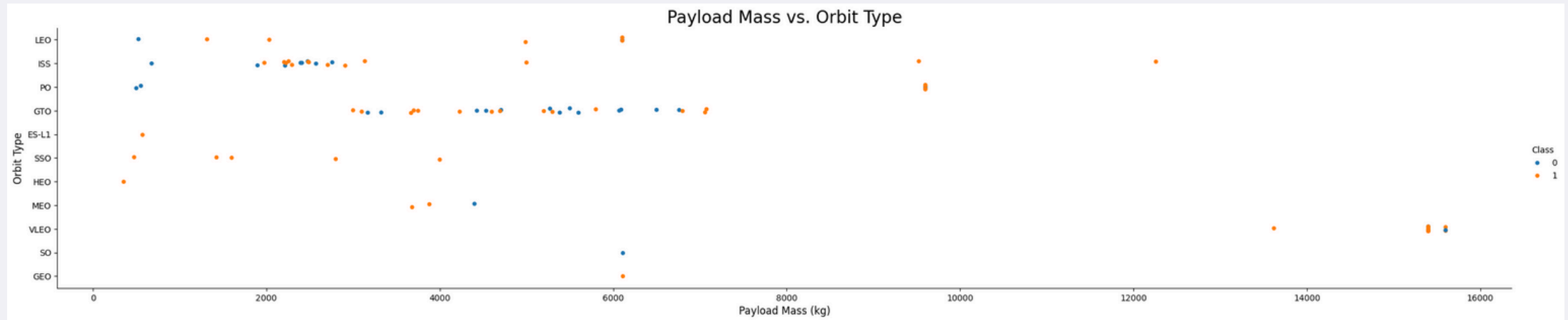


Flight Number vs. Orbit Type



In the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

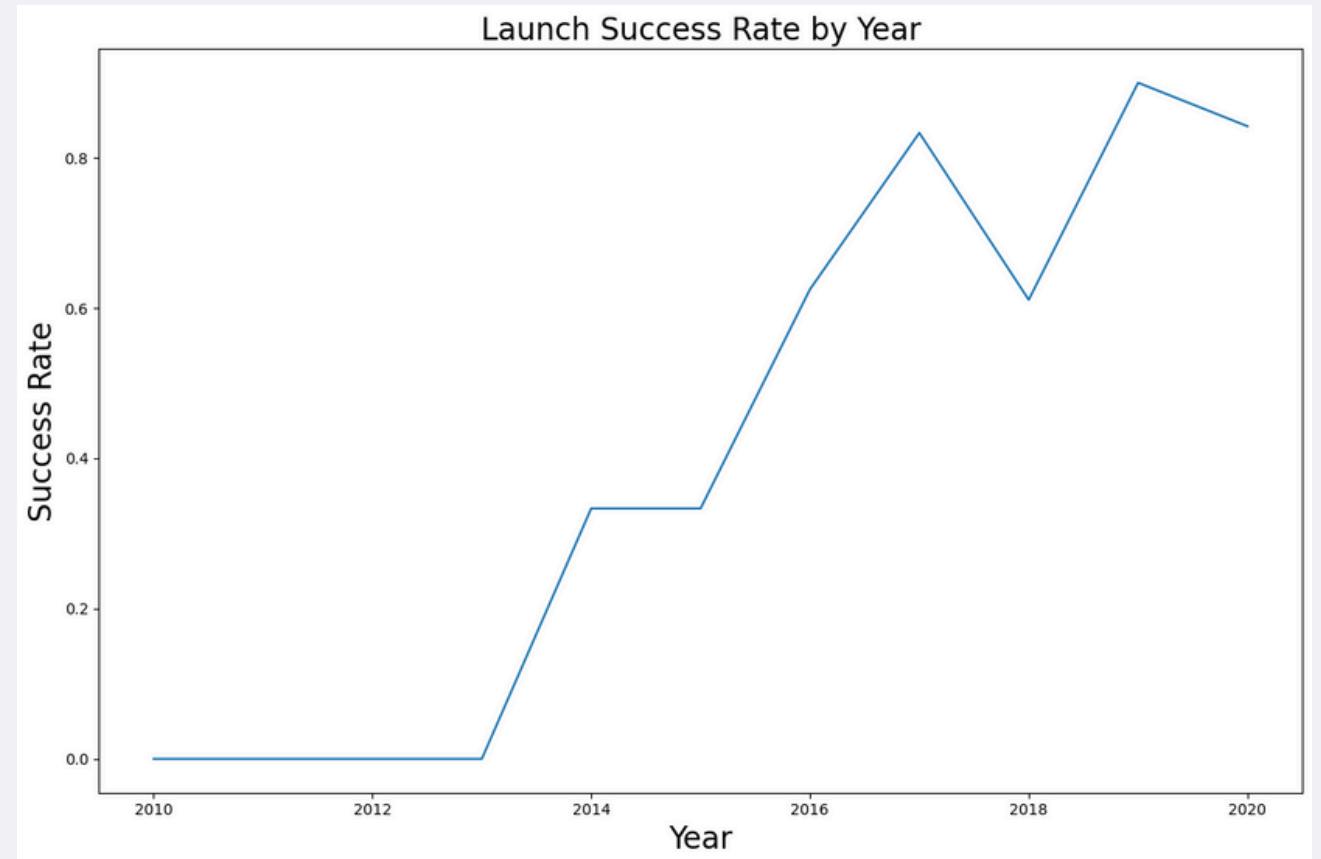
Payload vs. OrbitType



With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS. However for GTO we cannot distinguish this well as both positive landing rate and negative landing (unsuccessful mission) are both there here.

Launch Success Yearly Trend

The success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.



All Launch Site Names

We get the unique values of the Launch Sites by using “DISTINCT”

Display the names of the unique launch sites in the space mission

```
In [10]: %sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[10]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

Launch Site Names Begin with 'CCA'

We get only 5 rows by using “LIMIT”

Display 5 records where launch sites begin with the string 'CCA'

```
In [11]: %sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5
```

* sqlite:///my_data1.db
Done.

```
Out[11]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

We can get the total payload mass by calculating the sum of all values by using “SUM”

Display the total payload mass carried by boosters launched by NASA (CRS)

```
In [12]: %sql SELECT SUM(PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[12]: SUM(PAYLOAD_MASS_KG_)  
         45596
```

Average Payload Mass by F9 v1.1

We can get the average payload mass carried by booster version F9 v1.1 by calculating the average of all values by using “AVG”

Task 4

Display average payload mass carried by booster version F9 v1.1

```
In [14]: %sql SELECT AVG(PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE Booster_Version = 'F9 v1.1'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[14]: AVG(PAYLOAD_MASS_KG_)
```

```
2928.4
```

First Successful Ground Landing Date

We can find the dates of the first successful landing outcome on ground pad by using “MIN” since the first date is the same as the minimum date

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
In [15]: %sql SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[15]: MIN(Date)
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

The payload mass data was taken between 4000 and 6000, and the landing outcome was determined to be “success (drone ship)”

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

In [16]: `%sql SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS_KG_ > 4000 AND`

`* sqlite:///my_data1.db`

Done.

Out[16]: **Booster_Version**

F9 FT B1022

F9 FT B1026

F9 FT B1021.2

F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

We use “COUNT” to calculate the total number of successful and failure mission outcomes

List the total number of successful and failure mission outcomes

```
In [17]: %sql SELECT Mission_Outcome, COUNT(*) AS Count FROM SPACEXTABLE GROUP BY Mission_Outcome
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[17]:
```

Mission_Outcome	Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried MaximumPayload

We can get the maximum payload masses by using “MAX”

List all the booster_versions that have carried the maximum payload mass. Use a subquery.

```
In [20]: %sql SELECT DISTINCT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTA
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[20]: Booster_Version
```

F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

We can use `substr(Date, 6, 2)` as month to get the months and `substr(Date, 0, 5)='2015'` for year. We use “LIKE” to filter for `'%Failure (drone ship)%'`

Note: SQLite does not support monthnames. So you need to use `substr(Date, 6, 2)` as month to get the months and `substr(Date, 0, 5)='2015'` for year.

In [23]: `%sql SELECT substr(Date, 6, 2) as month, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE substr(Date, 0, 5)='2015' AND Landing_Outcome LIKE '%Failure (drone ship)%'`

* sqlite:///my_data1.db

Done.

Out[23]:

month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

We can use “COUNT” to count the landing outcomes, “BETWEEN” for the desired date range, and “ORDER” for ordering values in descending order

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

In [24]:

```
%sql SELECT Landing_Outcome, COUNT(Landing_Outcome) as Count FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' ORDER BY Count DESC
```

* sqlite:///my_data1.db

Done.

Out[24]:

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

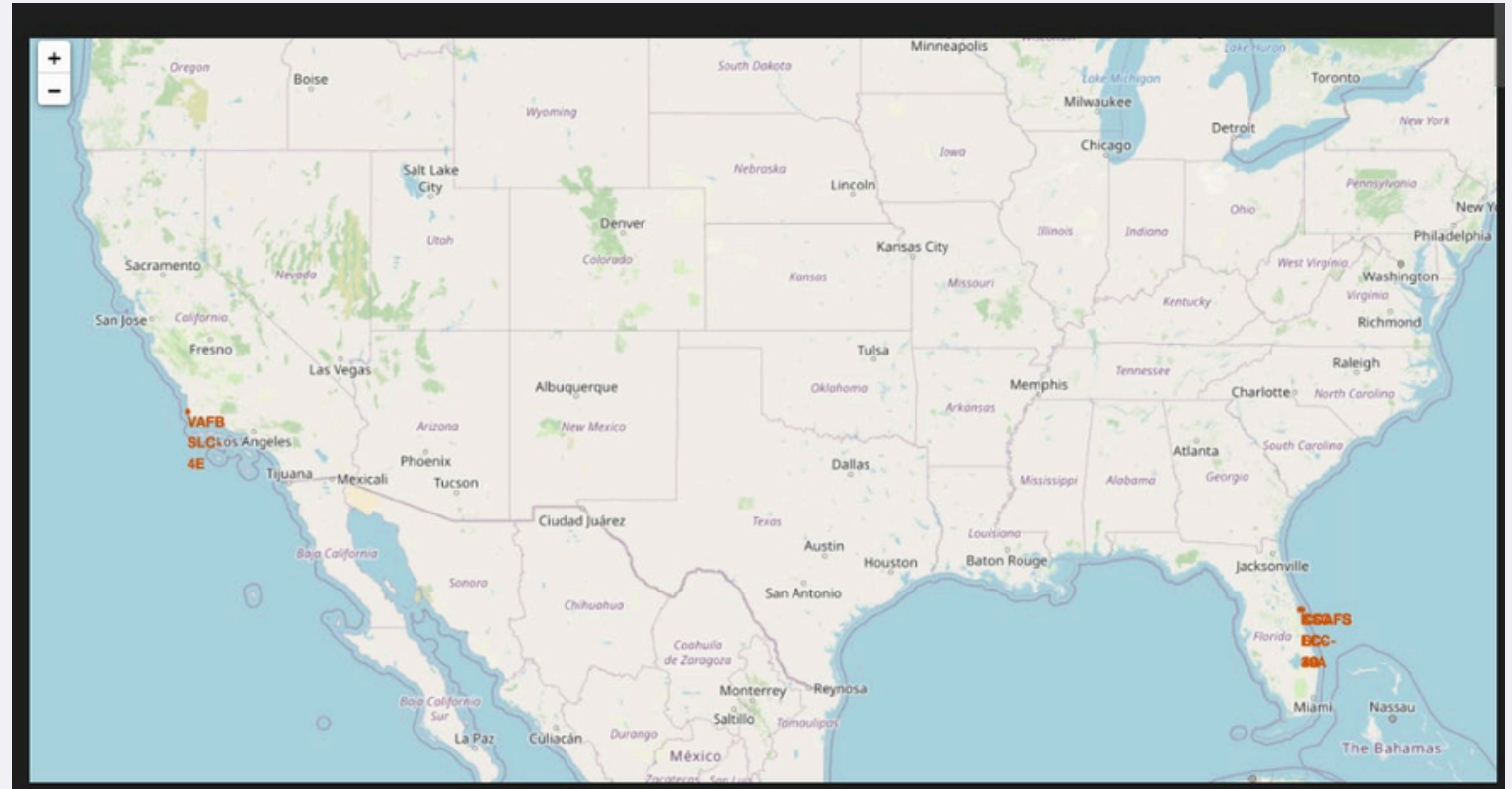
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a dark blue sky with stars and a view of the Earth's surface from space. The Earth's surface is mostly dark blue, with a thin layer of white clouds. A curved horizon line separates the dark sky from the Earth's surface. In the lower right, there are bright, glowing yellow and orange lights, likely representing city lights or a large fire. The overall image has a high-contrast, cinematic quality.

Section 3

Launch Sites Proximities Analysis

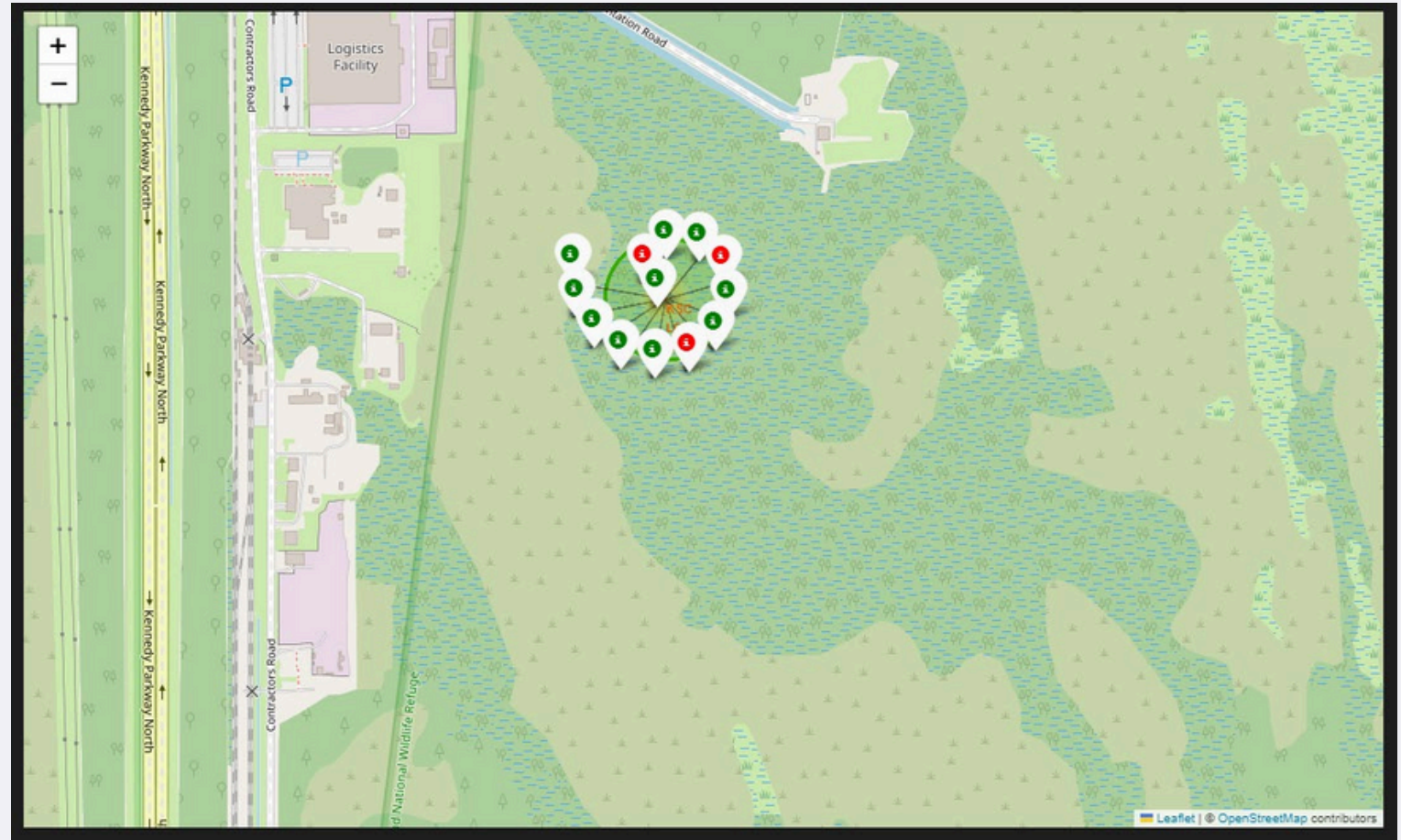
Global Launch Sites Overview

This map shows the geographic distribution of all recorded SpaceX launch sites. Locations are clustered near coasts to support booster recovery and range safety.



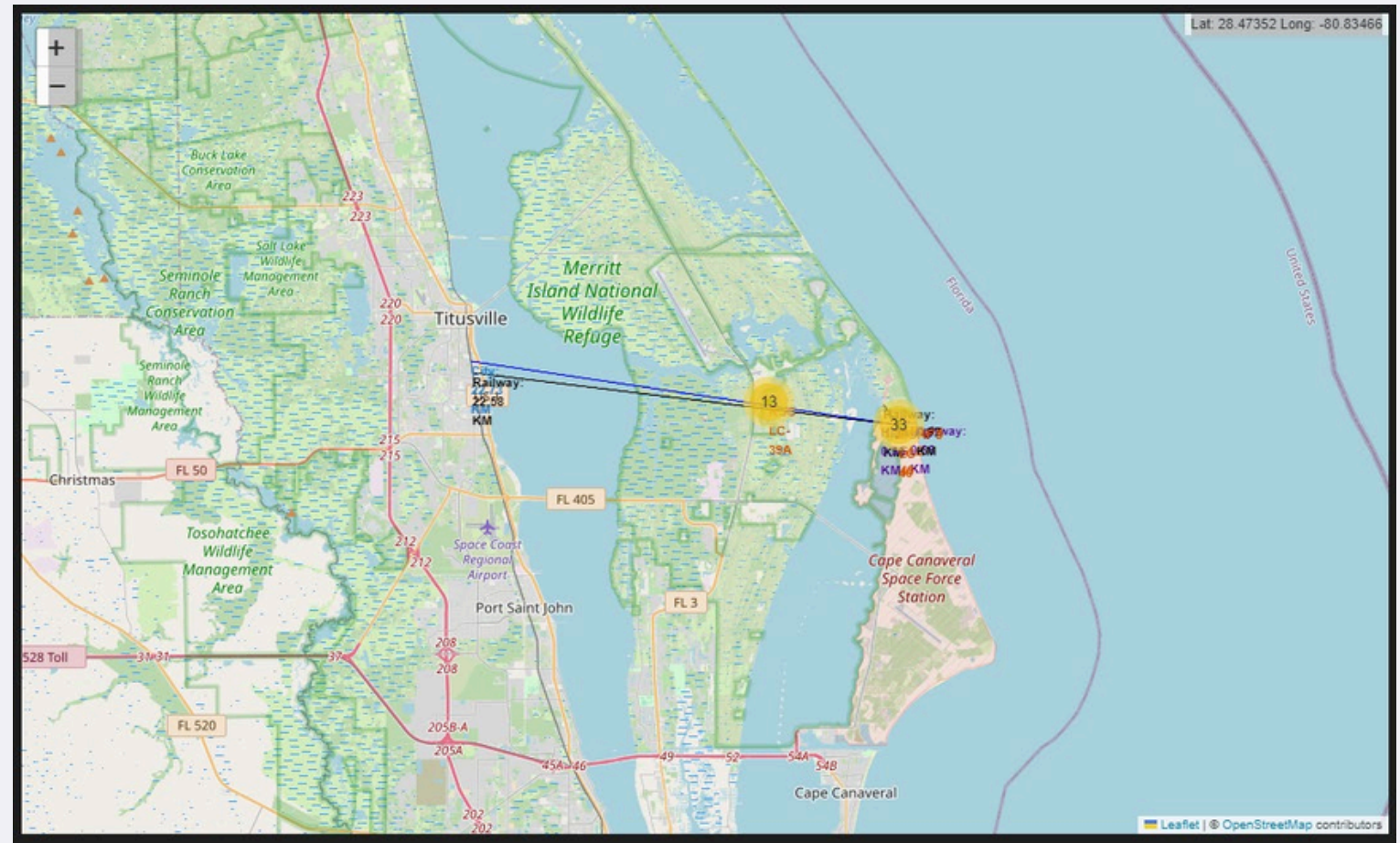
Launch Outcome by Site

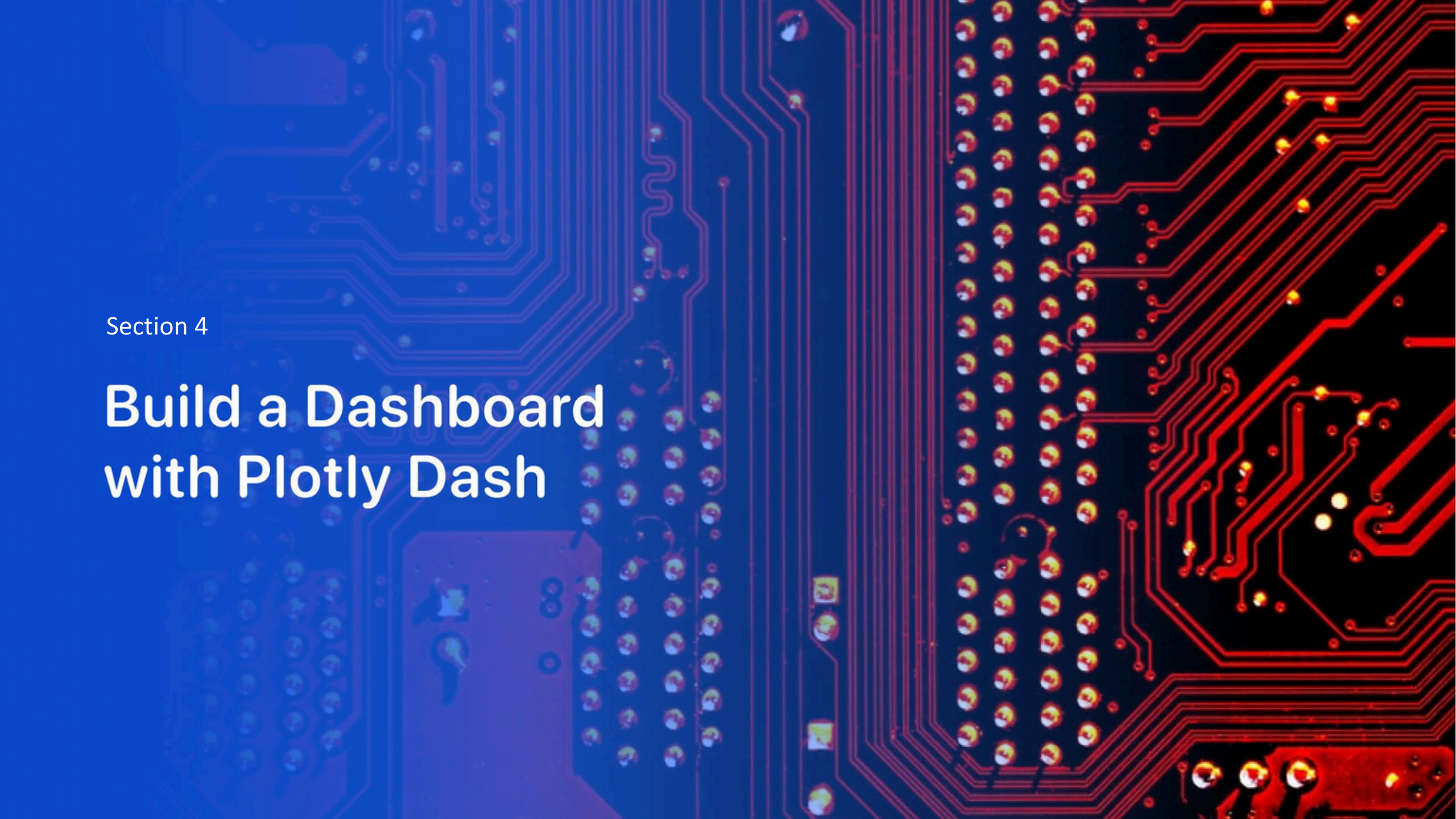
Color-coded markers provide a visual summary of success rates by site. The Kennedy Space Center (KSC LC-39A) shows predominantly successful missions.



Launch Site Proximity to Key Infrastructure

Proximity to transportation and recovery infrastructure supports SpaceX's reusability and logistics goals.



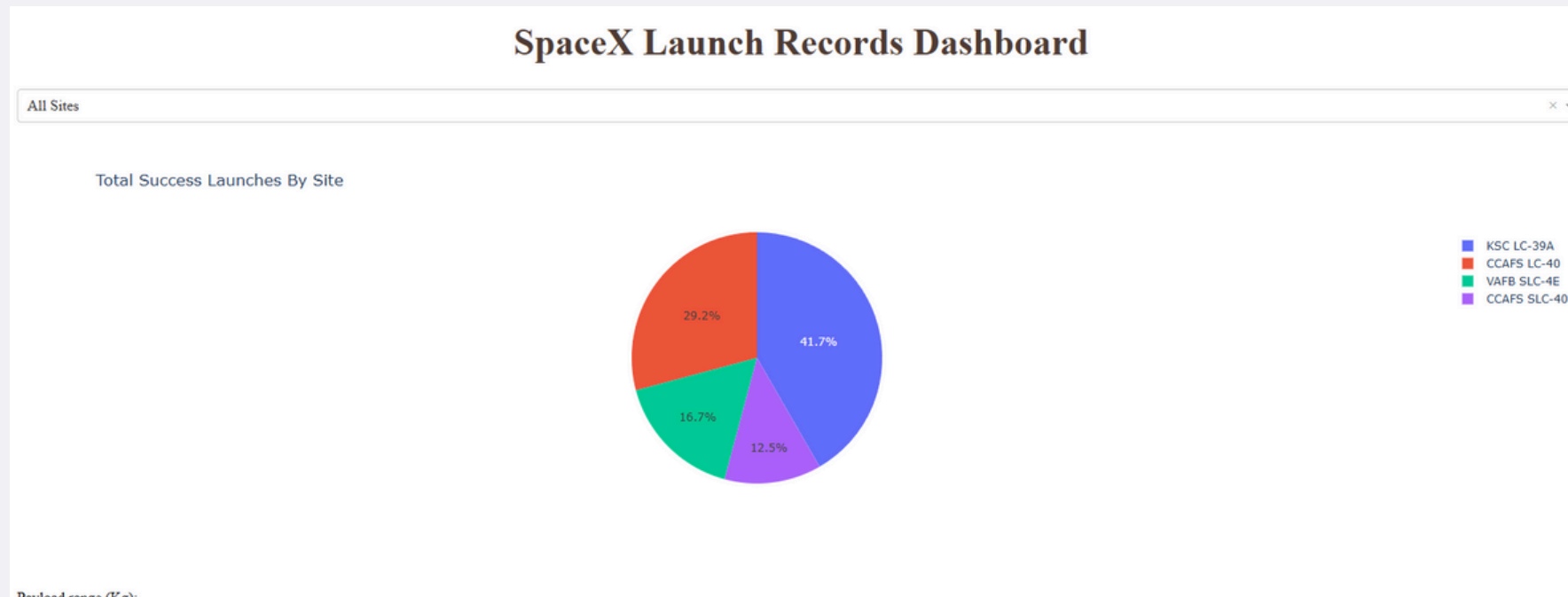


Section 4

Build a Dashboard with Plotly Dash

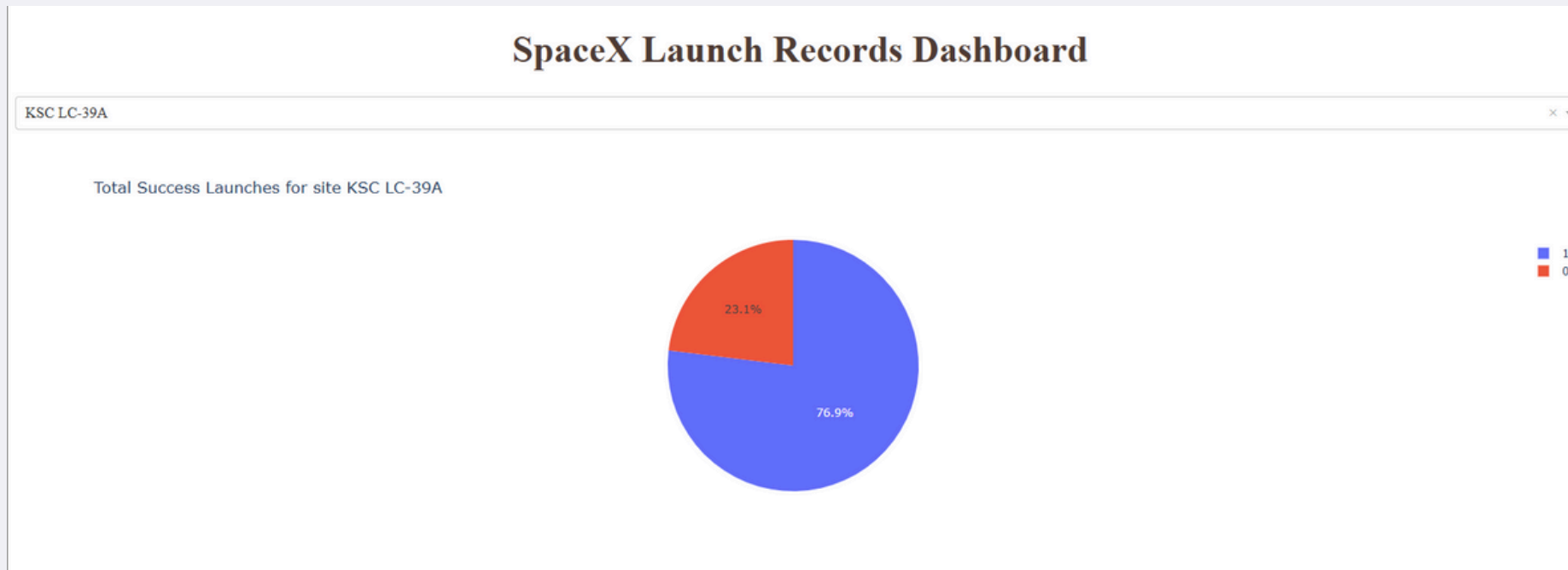
Launch Success Distribution – All Sites

The dashboard reveals launch success counts across all locations. Majority of launches were successful, with minor failures concentrated in earlier years.



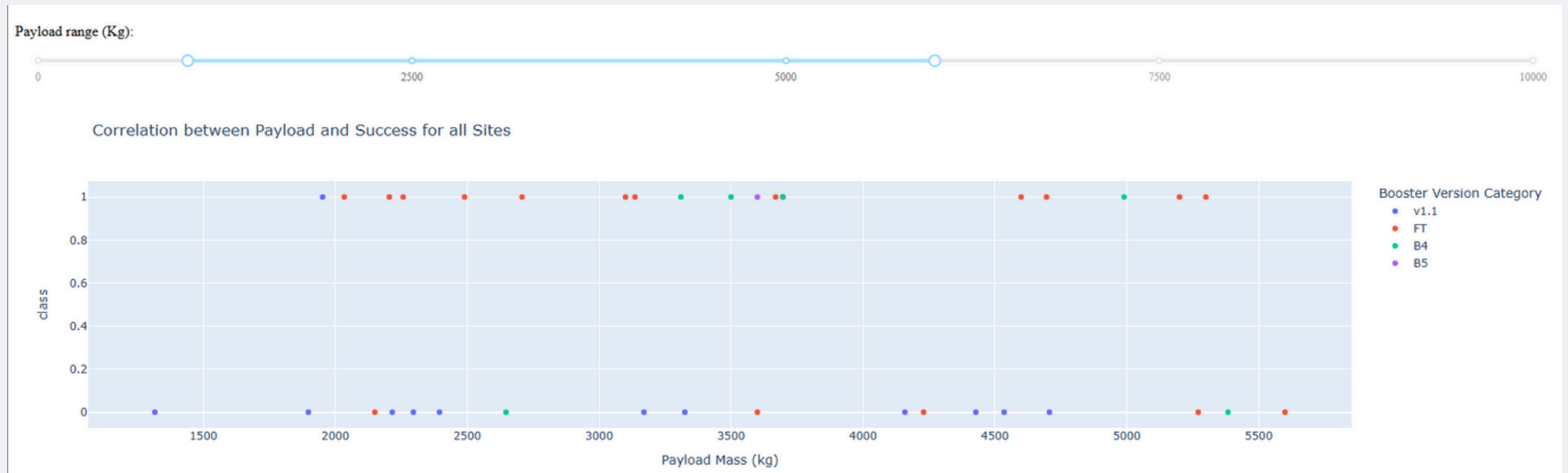
Top Performing Launch Site

KSC LC-39A demonstrates a high success ratio, suggesting it is a reliable site for mission-critical launches.



Payload Mass vs Outcome (Filtered View)

Payloads between 2000 kg and 5500 kg launched with FT boosters show a higher success rate.

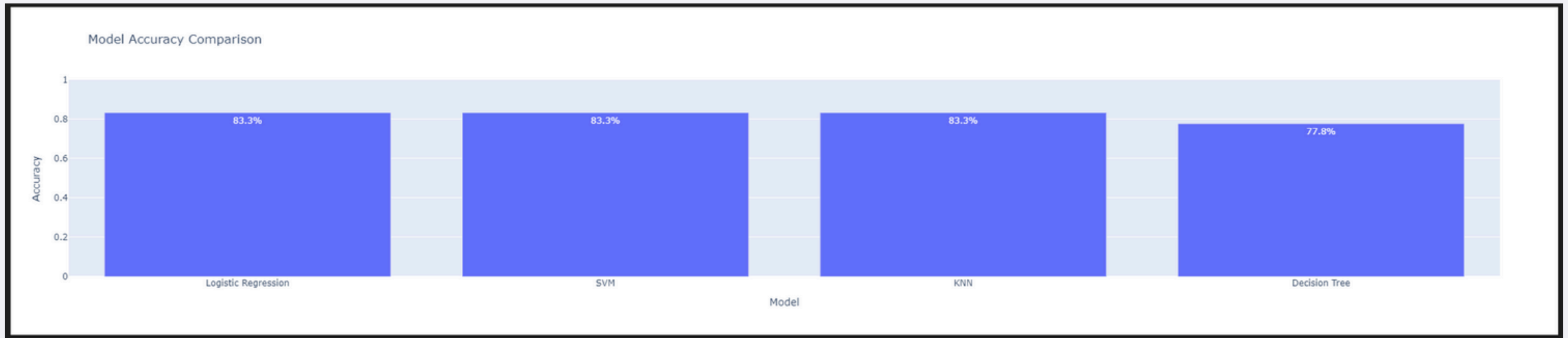




Section 5

Predictive Analysis (Classification)

Classification Accuracy

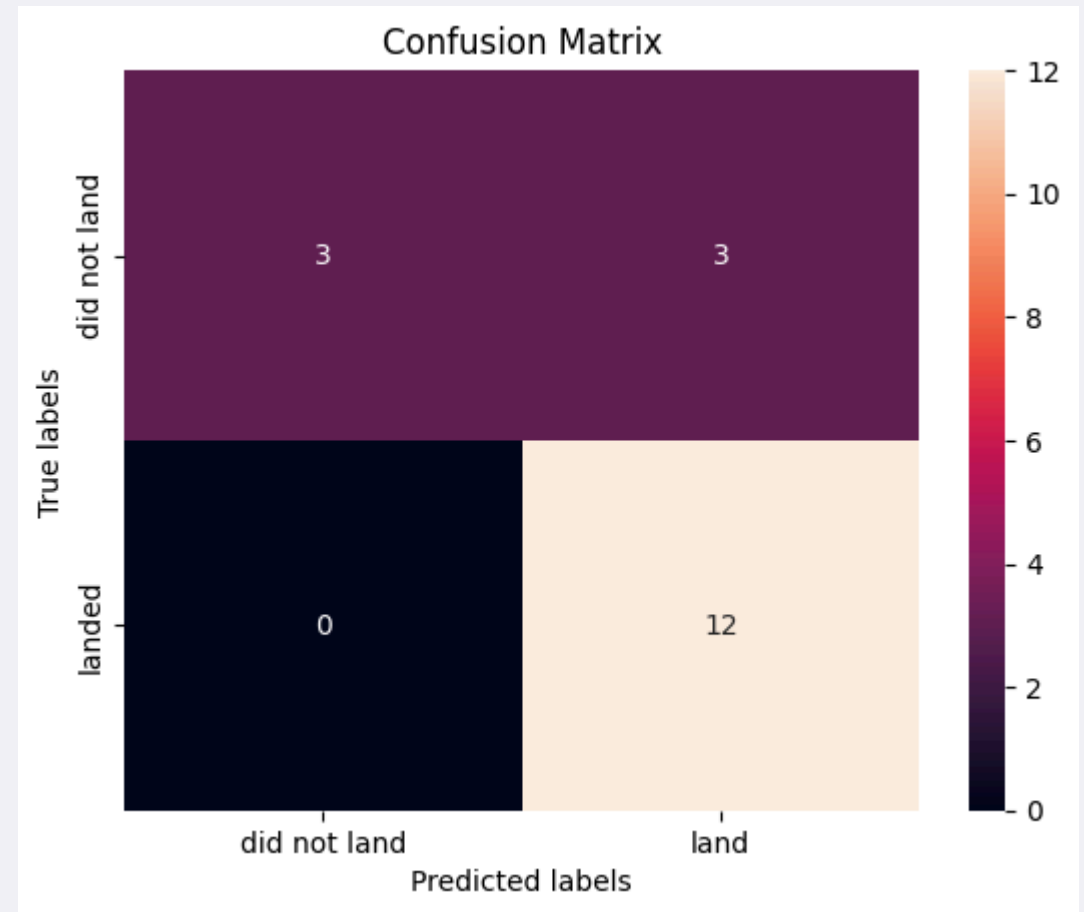


Confusion Matrix

Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

Overview:

- True Positive - 12 (True label is landed, Predicted label is also landed)
- False Positive - 3 (True label is not landed, Predicted label is landed)



Conclusions

1. The Kennedy Space Center LC-39A had the highest success ratio, making it the most reliable launch site.
2. Payload mass and orbit type influence success probability. Mid-weight payloads and LEO/MEO orbits had better outcomes.
3. Logistic Regression was the best-performing predictive model with an 83.3% accuracy.
4. Visual tools like Folium and Plotly Dash enabled intuitive understanding of launch outcomes and patterns.

Thank you!

